CS3400 - Tutorial 2

1. Why can an array provide O(1) random access while a linked list cannot?
2. Why does inserting into the middle of an array-based list require shifting?
3. Why does a linked list not require contiguous memory?
4. Why does a doubly linked list require more memory than a singly linked list?
5. Explain why a circular linked list cannot use nullptr to identify the end of the list.
6. Why is a circular queue preferable to repeatedly shifting elements in an array-based queue?
7. Explain the difference between an ADT and its implementation using the List ADT as an example
8. What is a memory leak?
9. What is a dangling pointer?
10. Why should a pointer generally be set to nullptr after deleting the object it points to?
11. Explain how a linked-list destructor prevents memory leaks.
12. Why must `current->next` be saved before deleting `current` during linked-list destruction?
13. What is RAII and why is it useful in C++ data structures?
14. What is the difference between shallow copying and deep copying?
15. Why can shallow copying a linked list cause a double deletion?
16. Why might a class containing a raw pointer require a custom destructor?
17. Explain the Rule of Three.
18. Implement a singly linked list with:

- `insertFront()`
- `insertBack()`
- `removeFront()`
- `remove()`
- `search()`
- `clear()`
- a destructor.

19. Implement a stack using an array.
20. Implement the same Stack ADT using a linked list.
21. Implement a queue using a circular array.
22. Implement a queue using a linked list with both `front` and `rear` pointers.
23. Implement a deque using a doubly linked list.
24. Suppose a linked list has one million elements. Is insertion necessarily O(1)? Explain.
25. A student says:

    "Linked lists are always faster than arrays for insertion."

    Is this statement correct? Explain the missing condition.

26. Consider:

    ```
    Node* p = new Node{10, nullptr};
    Node* q = p;
    delete p;
    ```

    What is the state of q after `delete p`?

27. Consider:

    ```
    int* p = new int[10];

    p = new int[20];

    delete[] p;
    ```

    Is there a memory leak? Explain.

28. Consider two linked lists created through shallow copying. Explain what happens if one list is destroyed.
29. Why can an array-based structure sometimes outperform a linked structure even when both operations have the same asymptotic complexity?
30. Design a List ADT and provide **two different implementations**. Explain why users of the ADT should not need to know which implementation is being used.